Guided Tutorial: Computer Vision and Deep Learning

🌟Learning Outcomes

By the end of this tutorial, you will be able to,

- Explain what Deep learning is and how it differs from traditional Machine Learning
- Identify key components of a Convolutional Neural Network (CNN)
- Perform a simple convolution and pooling operation
- Build and test a very small CNN on image data using Tensorflow/ Keras
- Prepare for the in-class Tutorial 10 exercises

🧠1. What is Deep Learning?

Deep Learning is a branch of Machine Learning that uses Artificial Neural Networks with multiple layers to automatically learn patterns and features from data such as images, sound, and text.

Traditional ML requires manual feature extraction using edge detection, corners and texture identification.

Deep Learning models learn these features automatically from raw pixels.

🎯Example:

- A traditional algorithm might detect a cat by checking for edges, shapes and colors
- A CNN learns what a cat looks like by itself after seeing many cat-labeled images.

👁️2. What is Computer Vision?

Computer Vision enables computers to “see” and interpret images or videos.

In Computer Vision, Deep Learning with the use of CNNs has largely replaced manual image feature engineering as CNNs can learn hierarchical features.

- **Layer 1:** Edges
- **Layer 2:** Shapes and textures
- **Layer 3:** Object parts and full objects

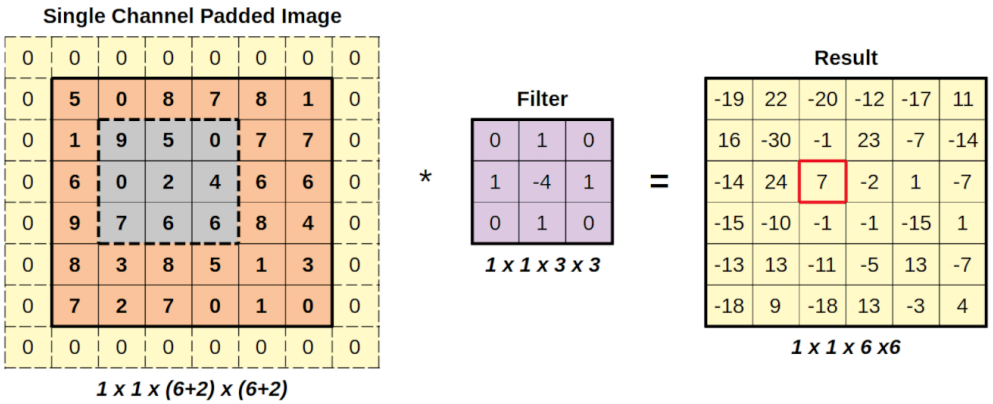
⚙️3. Understanding the CNN structure

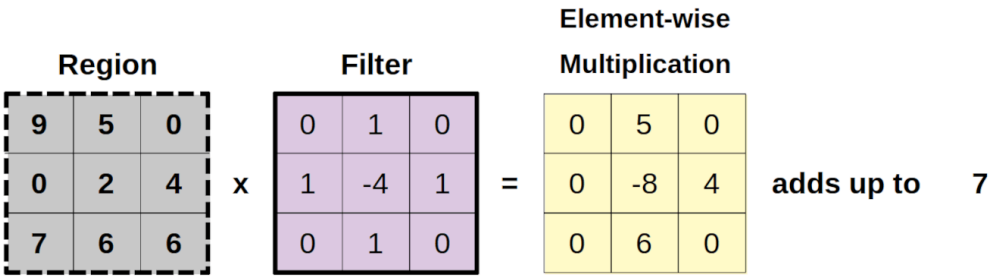
A typical CNN has 3 main types of layers:

Layer	Purpose	Example
Convolutional Layer	Extracts features using filters/kernels	Detect edges
Pooling Layer	Reduces size and complexity	Keep strongest features
Fully Connected Layer	Combines features for classification	Decide object class

📋4. Hands-On Activity: Convolution

Activity 1: Simulate a 3x3 kernel on a 5x5 grayscale image - Try in Colab





```python
import numpy as np
from scipy.signal import convolve2d
import matplotlib.pyplot as plt

# 5x5 input image - grayscale
image = np.array([[1,2,3,0,0],
                  [4,5,6,0,0],
                  [7,8,9,0,0],
                  [0,0,0,0,0],
                  [0,0,0,0,0]])

# 3x3 kernel (edge detector)
kernel = np.array([[1,0,-1],
                   [1,0,-1],
                   [1,0,-1]])

result = convolve2d(image, kernel, mode='valid')
print(result)

# Create a figure with 1 row and 2 columns of subplots
# The 'axs' variable will be an array of axes objects
fig, axs = plt.subplots(1, 2, figsize=(10, 5)) # figsize adjusts the
overall size of the figure

# Display the first image on the first subplot
axs[0].imshow(image, cmap='gray')
axs[0].set_title('Image 1')
axs[0].axis('off') # Turn off axis ticks and labels for cleaner image
display

# Display the second image on the second subplot
axs[1].imshow(result, cmap='gray')
axs[1].set_title('Image 2')
axs[1].axis('off')

# Adjust layout to prevent titles/labels from overlapping
plt.tight_layout()

# Show the figure
plt.show()
```

🌊5. Pooling: Simplifying the Feature Map - Try in Colab

Pooling reduces the image size while keeping the most important information.

Types:

- **Max Pooling:** Keep the largest value in the region
- **Average Pooling:** Takes the mean value

Activity 2: Try Max Pooling and Average Pooling

```
feature_map = np.array([[4,5,6,8],
                        [3,2,1,0],
                        [9,8,7,6],
                        [4,3,2,1]])

# 2x2 max pooling
max_pool = np.max(feature_map.reshape(2,2,2,2), axis=(2,3))
print("Max Pooling:\n", max_pool)

# 2x2 average pooling
avg_pool = np.mean(feature_map.reshape(2,2,2,2), axis=(2,3))
print("Average Pooling:\n", avg_pool)

fig, axs = plt.subplots(1, 3, figsize=(10, 5)) # figsize adjusts the
overall size of the figure

# Display the first image on the first subplot
axs[0].imshow(result, cmap='gray')
axs[0].set_title('Image 1')
axs[0].axis('off') # Turn off axis ticks and labels for cleaner image
display

# Display the second image on the second subplot
axs[1].imshow(max_pool, cmap='gray')
axs[1].set_title('Max Pool')
axs[1].axis('off')

axs[2].imshow(avg_pool, cmap='gray')
axs[2].set_title('Average Pool')
axs[2].axis('off')

# Adjust layout to prevent titles/labels from overlapping
plt.tight_layout()

# Show the figure
plt.show()
```

🤖6. Build the first CNN

Let's use Tensorflow/ Keras to create a very small CNN that classifies digits in MNIST dataset.

Activity 3: Build the model

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

# Load and normalize dataset
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data() #
download the mnist dataset and load training and testing sets
X_train = X_train[..., tf.newaxis] / 255.0 # Normalization
X_test = X_test[..., tf.newaxis] / 255.0

# Build CNN
 # Our model is a sequential model
  # layers.Conv2D - Define the convolutional layer with 8 filters, and 3x3
kernel
  # layers.MaxPooling2D - Define max-pooling layer of pool size 2
  # layers.Flatten - Flattens the input
  # layers.Dense - Densely connected layer
model = models.Sequential([
    layers.Conv2D(8, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy']) # Configures the model for training
model.fit(X_train, y_train, epochs=2, validation_data=(X_test, y_test)) #
Starts the training process
```

📊7. Visualizing the results

Activity 4: Plot the Accuracy Curve

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Model Performance')
plt.show()
```